



Search...

Courses

Tutorials

Practice

Jobs

3



Problem

Editorial

Submissions

Comments



Python3

Start Timer



Seating Arrangement



Difficulty: Easy

Accuracy: 49.85%

Submissions: 59K+

Points: 2

Given an integer **k** representing the number of people to be seated and an array **seats[]**, where **0** denotes an empty seat and **1** denotes an occupied seat.

Determine whether it is possible to seat all **k** people such that no two occupied seats are adjacent (including newly seated people).

Examples:

Input: k = 2, seats[] = [0, 0, 1, 0, 0, 0, 1]

Output: true

Explanation: The two people can sit at index 0 and 4.

Input: k = 1, seats[] = [0, 1, 0]

Output: false

Explanation: There is no way to get a seat for one person.

Input: k = 0, seats[] = [0, 0, 0, 1, 1]

Output: false

Explanation: The seating arrangement already contains two

```
1 class Solution:
2     def canSeatAllPeople(self, k, seats):
3         n = len(seats)
4
5         # Check if current arrangement is already invalid
6         for i in range(n - 1):
7             if seats[i] == 1 and seats[i + 1] == 1:
8                 return False
9
10        count = 0
11
12        for i in range(n):
13            if seats[i] == 0:
14                left_empty = (i == 0) or (seats[i - 1] == 0)
15                right_empty = (i == n - 1) or (seats[i + 1] == 0)
16
17                if left_empty and right_empty:
18                    seats[i] = 1
19                    count += 1
20
21        return count >= k
```

[Custom Input](#)[Compile & Run](#)[Submit](#)